Assignment-9.1

2303A51680

K Rishitha

Batch-23

Problem 1:
Consider the following Python function:
def find_max(numbers):
return max(numbers)
Task:
• Write documentation for the function in all three formats:
(a) Docstring
(b) Inline comments
(c) Google-style documentation
• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

```python
#docstring
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
    Parameters:
    numbers (list): A list of numbers (integers or floats).
    Returns:
    int or float: The maximum number in the list.
    """
    return max(numbers)
print(find_max.__doc__)
# Inline comments
def find_max(numbers):
    # Use the built-in max function to find the maximum number in the list
    return max(numbers) # This line returns the maximum number found in the list of numbers provided a
print(find_max.__doc__)
#Google style documentation
def find_max(numbers):
    """
    Find and return the maximum number in a list of numbers.
    Args:
        numbers (list): A list of numbers (integers or floats).
    Returns:
        int or float: The maximum number in the list.
    Raises:
        ValueError: If the input list is empty.
        TypeError: If the input is not a list of numbers.
    """
```

```python
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list of numbers.")
    if not numbers:
        raise ValueError("Input list cannot be empty.")
    return max(numbers)
print(find_max.__doc__)
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/
Docstring version: 99
Inline comment version: 99
Google-style version: 99
PS C:\Users\HP\Desktop\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/

Find and return the maximum number in a list of numbers.
Parameters:
numbers (list): A list of numbers (integers or floats).
Returns:
int or float: The maximum number in the list.

None

Find and return the maximum number in a list of numbers.
Args:
    numbers (list): A list of numbers (integers or floats).
Returns:
    int or float: The maximum number in the list.
```

Problem 2:

Consider the following Python function:

def login(user, password, credentials):

return credentials.get(user) == password

Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

```python
#docstring
def login(user, password,credentials):
    """Logs in a user if the provided credentials are correct.
    Args:
        user (str): The username of the user trying to log in.
        password (str): The password of the user trying to log in.
        credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    Returns:
        bool: True if the login is successful, False otherwise.
        """
    return credentials.get(user) == password
print(login.__doc__)
# Inline comments
def login(user, password,credentials):
    # Check if the provided username exists in the credentials dictionary and if the corresponding pas
    return credentials.get(user) == password # This line checks if the username exists in the credenti
print(login.__doc__)
#Google style documentation
def login(user, password,credentials):
    """
    Logs in a user if the provided credentials are correct.
    Args:
        user (str): The username of the user trying to log in.
        password (str): The password of the user trying to log in.
        credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    Returns:
        bool: True if the login is successful, False otherwise.
    Raises:
```

```python
        bool: True if the login is successful, False otherwise.
    Raises:
        TypeError: If user or password is not a string, or if credentials is not a dictionary.
    """
    if not isinstance(user, str) or not isinstance(password, str):
        raise TypeError("User and password must be strings.")
    if not isinstance(credentials, dict):
        raise TypeError("Credentials must be a dictionary.")
    return credentials.get(user) == password # This line checks if the username exists in the credenti
print(login.__doc__)
```

```
PS C:\Users\HP\Desktop\AI-Lab> & C:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe c:/Users/HP/
ion.py
Logs in a user if the provided credentials are correct.
Args:
    user (str): The username of the user trying to log in.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
Logs in a user if the provided credentials are correct.
Args:
    user (str): The username of the user trying to log in.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
Args:
    user (str): The username of the user trying to log in.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    user (str): The username of the user trying to log in.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    password (str): The password of the user trying to log in.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
    credentials (dict): A dictionary containing valid usernames and their corresponding passwords.
Returns:
Returns:
    bool: True if the login is successful, False otherwise.
```

Problem 3:

Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:

o add(a, b) – returns the sum of two numbers

o subtract(a, b) – returns the difference of two numbers

o multiply(a, b) – returns the product of two numbers

o divide(a, b) – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

calculator.py > divide

```python
def add(a, b):
    """
    Return the sum of a and b.
    Parameters:
        a (int or float): The first number to add.
        b (int or float): The second number to add.
    Returns:
        int or float: The sum of a and b.
    """
    return a + b
def subtract(a, b):
    """Return the difference of a and b.
    Parameters:
        a (int or float): The number from which to subtract.
        b (int or float): The number to subtract.
    """
    return a - b
def multiply(a, b):
    """Return the product of a and b.
    Parameters:
        a (int or float): The first number to multiply.
        b (int or float): The second number to multiply.
    """
    return a * b
def divide(a, b):
    """Return the quotient of a and b. Raises ValueError if b is zero
    parameters:
        a (int or float): The number to be divided.
        b (int or float): The number to divide by
    Returns:
        int or float: The quotient of a and b
    Raises:
```

```python
        int or float: The quotient of a and b
    Raises:
        ValueError: If b is zero, as division by zero is undefined
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b
```

```
py
PS C:\Users\HP\Desktop\AI-Lab> python -m pydoc -w calculator
wrote calculator.html
wrote calculator.html
PS C:\Users\HP\Desktop\AI-Lab>
```

```python
def add(a, b):
    """
    Return the sum of a and b.
    Parameters:
        a (int or float): The first number to add.
        b (int or float): The second number to add.
    Returns:
        int or float: The sum of a and b.
    """
    return a + b
def subtract(a, b):
    """Return the difference of a and b.
    Parameters:
        a (int or float): The number from which to subtract.
        b (int or float): The number to subtract.
    """
    return a - b
def multiply(a, b):
    """Return the product of a and b.
    Parameters:
        a (int or float): The first number to multiply.
        b (int or float): The second number to multiply.
    """
    return a * b
def divide(a, b):
    """Return the quotient of a and b. Raises ValueError if b is zero
    parameters:
        a (int or float): The number to be divided.
        b (int or float): The number to divide by
    Returns:
        int or float: The quotient of a and b
    Raises:
        ValueError: If b is zero, as division by zero is undefined
    """
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    return a / b
```

Problem 4:

Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

conversion.py > binary_to_decimal

```python
def decimal_to_binary(n):
    """Convert a decimal number to binary.
    Parameters:
        n (int): The decimal number to convert.
    Returns:
        str: The binary representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    binary = ""
    if n == 0:
        return "0"
    while n > 0:
        binary = str(n % 2) + binary
        n //= 2
    return binary
def binary_to_decimal(b):
    """Convert a binary number to decimal.
    Parameters:
        b (str): The binary number to convert.
    Returns:
        int: The decimal representation of the binary number.
    """
    if not all(char in '01' for char in b):
        raise ValueError("Input must be a binary string containing only '0' and '1'.")
    decimal = 0
    for index, digit in enumerate(reversed(b)):
        decimal += int(digit) * (2 ** index)
    return decimal
def decimal_to_hexadecimal(n):
    """Convert a decimal number to hexadecimal.
    Parameters:
        n (int): The decimal number to convert.
    Returns:
        str: The hexadecimal representation of the decimal number.
```

```python
    Returns:
        str: The hexadecimal representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    if n == 0:
        return "0"
    while n > 0:
        hexadecimal = hex_digits[n % 16] + hexadecimal
        n //= 16
    return hexadecimal
```

File C:/Users/HP/Desktop/AI-Lab/conversion.py

```
lef decimal_to_binary(n):
    """Convert a decimal number to binary.
    Parameters:
        n (int): The decimal number to convert.
    Returns:
        str: The binary representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    binary = ""
    if n == 0:
        return "0"
    while n > 0:
        binary = str(n % 2) + binary
        n //= 2
    return binary
lef binary_to_decimal(b):
    """Convert a binary number to decimal.
    Parameters:
        b (str): The binary number to convert.
    Returns:
        int: The decimal representation of the binary number.
    """
    if not all(char in '01' for char in b):
        raise ValueError("Input must be a binary string containing only '0' and '1'.")
    decimal = 0
    for index, digit in enumerate(reversed(b)):
        decimal += int(digit) * (2 ** index)
    return decimal
lef decimal_to_hexadecimal(n):
    """Convert a decimal number to hexadecimal.
    Parameters:
        n (int): The decimal number to convert.
    Returns:
        str: The hexadecimal representation of the decimal number.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    if n == 0:
```

Problem 5

Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

course.py > remove_course

```python
def add_course(course_id, name, credits):
    """Adds a course to the course list.
    Parameters:
        course_id (str): The unique identifier for the course.
        name (str): The name of the course.
        credits (int): The number of credits for the course.
    Returns:
        dict: A dictionary representing the added course.
    """
    course = {
        "course_id": course_id,
        "name": name,
        "credits": credits
    }
    return course
def remove_course(course_id):
    """Removes a course from the course list.
    Parameters:
        course_id (str): The unique identifier for the course to be removed.
    Returns:
        bool: True if the course was successfully removed, False otherwise.
    """
    # This function would typically interact with a database or a list of courses to remove the specified course.
    # For demonstration purposes, we will assume the course is always successfully removed.
    return True
def get_course(course_id):
    """Retrieves a course from the course list.
    Parameters:
        course_id (str): The unique identifier for the course to be retrieved.
    Returns:
        dict: A dictionary representing the retrieved course, or None if the course is not found.
    """
    # This function would typically interact with a database or a list of courses to retrieve the specified course.
    # For demonstration purposes, we will return a sample course if the course_id matches "CS101", and None otherwise.
    if course_id == "CS101":
```

```python
    # This function would typically interact with a database or a list of courses to retrieve the specified course.
    # For demonstration purposes, we will return a sample course if the course_id matches "CS101", and None otherwise.
    if course_id == "CS101":
        return {
            "course_id": "CS101",
            "name": "Introduction to Computer Science",
            "credits": 3
        }
    else:
        return None
```

```
PS C:\Users\HP\Desktop\AI-Lab>
PS>python -m pydoc -w course
wrote course.html
PS C:\Users\HP\Desktop\AI-Lab>
```

Functions

add_course(course_id, name, credits)

Adds a course to the course list.
Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.
Returns:
dict: A dictionary representing the added course.

get_course(course_id)

Retrieves a course from the course list.
Parameters:
course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary representing the retrieved course, or None if the course is not found.

remove_course(course_id)

Removes a course from the course list.
Parameters:
course_id (str): The unique identifier for the course to be removed.
Returns:
bool: True if the course was successfully removed, False otherwise.